Question 2: Scaling the Tracker Analysis to 30,000 Objects and 100 Trackers

Mission Design Engineer Assignment | Karishma T. | Timed on a 12-thread Core i5-1235U laptop; "est." = estimate

1. Problem size and baseline

N_o = 30,000 objects, N_t = 100 trackers, P = $N_o N_t$ = 3x10^6 pairs, T = 86,400 s. The Question 1 script takes **9.1 s** for one pair (mean of 3 runs); 6.4 s of that is one-time start-up (JVM 0.6, imports 0.4, ITRF frame 4.4, Sun ephemeris 1.2, gravity field 0.3 s). Repeated in one process, a pair costs **1.40 s**.

Q1 stage (one pair)	s	class	Q1 stage (one pair)	s	class
JVM + imports + setup	6.45	fixed	range sampling + Brent	0.46	per pair
tracker propagation + FOV	0.83	0.04 trk / pair	off-boresight table	0.46	validation
EclipseDetector (+ cyl. check)	0.70	per object	altitude + Earth-blocked	0.18	validation

- One process per pair would take **317 days**. Looping the Q1 code in one process takes **49 days**: it re-propagates every tracker 30,000 times and recomputes every eclipse 100 times.
- A 1 s grid for every object is 30000 x 86,401 x 3 x 8 B = **62 GB**, and any fixed grid misses short events: in Question 1 a fixed 1 s check missed the 0.18 s crossing.

2. Cost model and run time

$$T_{total} = C_{fixed} + N_o \cdot c_{obj} + N_t \cdot c_{trk} + P \cdot c_{pair} \quad \text{(only } c_{pair} \text{ is multiplied by } 3\text{x}10^6\text{)}$$

Term / stage	Before (1 core)	After (1 core)	After (16 cores)	Basis
C_{fixed} (JVM, ITRF, DE, gravity)	6.4 s	6.4 s	6.4 s	measured, per worker
c_{trk} x 100 (tracker + ephemeris)	1.3e5 s	6.6 s	0.4 s	measured, 66 ms each
c_{obj} x 30,000 (SGP4 grid, eclipse)	4.6e4 s	478 s	30 s	measured, 0.50 + 15.4 ms each
Screening, all pairs, 60 s grid	none	465 s	29 s	measured, full 24 h
Refinement (1.6e7 windows)	2.3e6 s	3.7e4 s	2.3e3 s	est., 2.0 ms per window
Diagnostics (validation only)	1.7e6 s	sampled	sampled	measured, 0.57 s per pair (41%); run on a sample
Total	**4.2e6 s = 49 d**	**3.7e4 s = 10.4 h**	**1.1 h**	**112x faster; 16-core: Amdahl, ideal 0.7 h**

3. Program structure

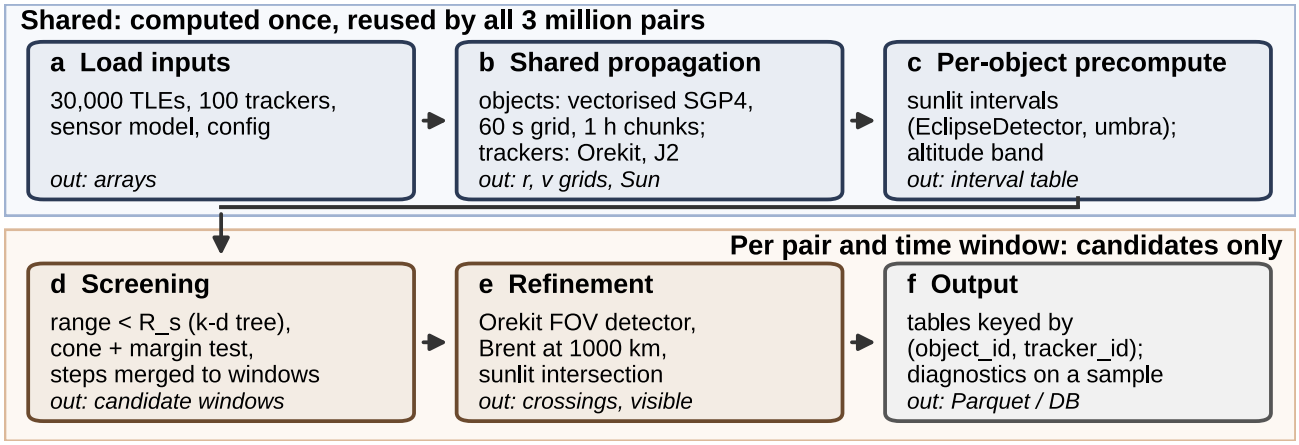


Diagram 1. Pipeline. Shared stages (top) run once and are reused by all pairs. Per-window stages (bottom) run per tracker and 1 h time chunk, on candidate windows only. Workers are time-chunk x tracker-block tasks on Ray or PySpark, one JVM each.

- **a, b, c: shared.** Objects go through python-sgp4 `SatrecArray` on a 60 s grid in 1 h chunks (30,000 x 61 states in 0.64 s). Trackers: Orekit `NumericalPropagator` (J2) with `getEphemerisGenerator()`.

Sunlit intervals depend only on the object, so all 100 trackers reuse them.

- **d, e, f: per window.** Screening: range test (k-d tree) and cone-plus-margin test, boresight from the tracker velocity. Refinement: Orekit FOV detector with the Q1 adaptive check interval (the 0.18 s graze is resolved), Brent at 1000 km, intersection with the sunlit intervals.

4. Efficiency strategies

Reduce the number of samples and pairs

- **Grid step.** A pass within R_0 = 1000 km shows on the grid within $R_s = R_0 + v_{max}\Delta t/2$ (v_{max} = 15.5 km/s, assumed). The cost $K(\Delta t) \propto (T/\Delta t)\cdot R_s^3$ is smallest at $\Delta t^* = R_0/v_{max}$ = 65 s with R_s = 1.5 R_0, so I use 60 s and R_s = 1465 km. Hermite interpolation error $\leq (\Delta t^4/384)\cdot n^4 r$ = 0.35 m, far below the ~1 km TLE error.
- **Candidates.** At $\rho = 2\text{x}10^{-8}$ km^{-3} (est., uniform shell) a tracker has $(4/3)\pi R_s^3\rho$ = 263 objects inside R_s, not 30,000. Synthetic catalogue: 308, at 8 ms per step (k-d build + 100 queries). 99.7% of pair-steps are discarded for visible events, yet 38% of pairs never get within R_s: the saving is in time, not pair pruning.
- **Angular skip bound** (crossings, no range limit). $|d\theta/dt| \leq v_{rel}/\rho + n_t$ gives $\tau = (\theta - 15°)/\omega$ before a pair can reach the cone. Simulated: only **3.4x** fewer checks (LEO pairs close fast), so I do not rely on it.

Make each sample cheap

- **Python-Java overhead.** One Orekit call from Python costs 5 to 15 µs, a vectorised SGP4 state 0.35 µs. The 43 million coarse states take 220 s through Orekit and 16 s through python-sgp4, so Orekit handles the trackers (about 2 s of calls), eclipse and refinement only. The boresight is the velocity vector (TNW); no `SpacecraftState` is built per sample.

Run in parallel

- **Partitioning.** Tasks are a time chunk (24 x 1 h) times a tracker block, so one SGP4 pass per chunk (0.64 s) serves every tracker in it. Chunks overlap by one step and windows are merged at the edges. Memory: whole coarse grid 1.04 GB, one 1 h chunk 88 MB. Each worker pays the 6.4 s start-up once. Amdahl: $S = 1/((1-p) + p/N)$ = **9.1** for p = 0.95, N = 16.

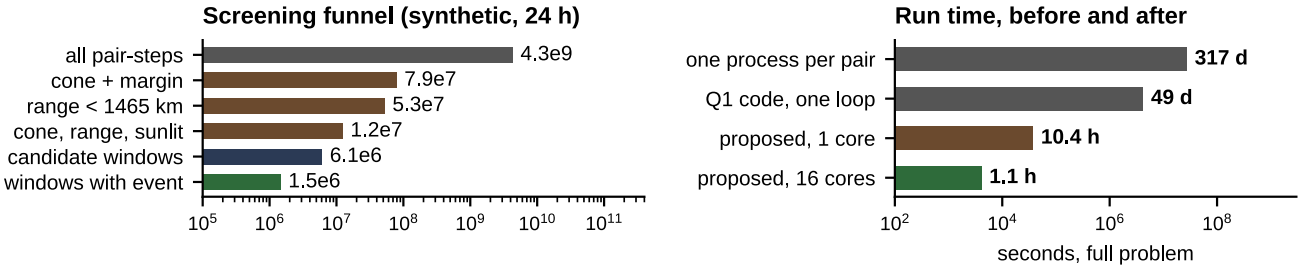


Diagram 2. Left: pair-time samples that survive each filter, from a synthetic catalogue (hit rate from 300 sampled windows checked on a 0.02 s grid). Right: wall time for the full problem.

5. Assumptions, safeguards and alternatives

- **Assumptions.** Q1 carries over (J2-only trackers, SGP4 objects, umbra-only shadow, 15° cone along the velocity). The real catalogue was not available: all counts come from a synthetic clustered LEO catalogue and 100 sun-synchronous 500 km trackers (cylindrical shadow). v_{max} and ρ are assumptions.
- **Safeguards.** R_s is conservative given v_{max}; the angular margin is a heuristic (range can shrink inside a step), so completeness is not proven. First test to run: a brute-force fine-grid comparison on a sample of pairs (not done).
- **Where the time goes.** Refinement is 97% of the optimised time: 1.6e7 candidate windows per day (an estimated 9.2e6 crossings and 1.5e6 visible events). Batched NumPy refinement is the next step; it is not implemented or measured.
- **Alternatives not chosen.** Brute force on a 1 s grid costs 2.1 h on one core but misses sub-second events (a 0.02 s grid: about 104 h). Frustum culling saves little at 25 ns per pair. A Hoots-style apogee/perigee sieve stays as a prefilter.